Thank you for taking the Week 10 : Assignment 10.

Course outline

About NPTEL ()

How does an NPTEL online course work? ()

Week 1 ()

Week 2 ()

Week 3 ()

week 4 ()

Week 5 ()

Week 6 ()

Week 7 ()

Week 8 ()

Week 9 ()

Week 10 ()

FLAMES - Part 01 (unit?unit=214&lesson=215)

FLAMES - Part 02 (unit?

Week 10 : Assignment 10

Your last recorded submission was on 2026-04-01, 21:30 IST Due date: 2026-04-01, 23:59 IST.

Relationship Prediction Using Character Elimination Logic

Background

A small social-logic application is designed to predict the **type of relationship** between two individuals based on their names. The program does not rely on predefined rules about people; instead, it uses a **character elimination strategy** followed by a **cyclic counting process** to arrive at a final outcome.

This kind of logic-based application is often used to help learners understand:

- list manipulation
- nested loops
- conditional branching
- iterative elimination logic
- modular arithmetic

Problem Context

The system asks two users to enter their names. To ensure fairness and consistency:

- All characters are converted to lowercase
- Spaces are removed
- Each name is converted into a list of characters

The program then compares both names character by character.

Core Logic Explained

1. Character Matching and Removal

- The program scans both character lists.
- When a **matching character** is found:

Assessment submitted.
X

unit=214&lesson=216)
FLAMES - Part 03 (unit?unit=214&lesson=217)
FLAMES - Part 04 (unit?unit=214&lesson=218)
FLAMES - Part 05 (unit?unit=214&lesson=219)
FLAMES - Part 06 (unit?unit=214&lesson=220)
Data Compression - Part 01 (unit?unit=214&lesson=221)
Data Compression - Part 02 (unit?unit=214&lesson=222)
Data Compression - Part 03 (unit?unit=214&lesson=223)
Data Compression - Part 04 (unit?unit=214&lesson=224)
Data Compression - Part 05 (unit?unit=214&lesson=225)
Week 10 : Programming Assignment 1 (/noc26_cs84/progassignment?name=643)
Week 10 : Programming Assignment 2

- That character is removed from **both lists**
- The process restarts immediately

- This continues until **no common characters remain**

This step ensures only **unmatched characters** contribute to the final calculation.

2. Counting Remaining Characters

After all matching letters are removed:

- The total number of remaining characters is calculated
- This count becomes the key value used to determine the relationship

3. Relationship Categories

The application uses a fixed list of relationship outcomes:

```
["Friends", "Love", "Affection", "Marriage", "Enemy", "Siblings"]
```

Using the remaining character count:

- A circular elimination process is applied
- One relationship is removed at each step
- The list keeps shrinking until one final relationship remains

This mirrors a **Josephus-style elimination pattern**, commonly used in algorithmic problems.

Assessment submitted.
X

(/noc26_cs84/progassignment?name=644)

- Week 10 : Programming Assignment 3 (/noc26_cs84/progassignment?name=646)
- Week 10 Feedback Form: The Joy of Computing using Python (unit?unit=214&lesson=226)
- **Quiz: Week 10 : Assignment 10 (assessment?name=660)**

Week 11 ()

Text Transcripts ()

Download Videos ()

```python
def remove_matching_letter(list1, list2):
    for i in range(len(list1)):
        for j in range(len(list2)):
            if list1[i] == list2[j]:
                list1.remove(list1[i])
                list2.remove(list2[j])
                return [list1 + ['*'] + list2, True]
    return [list1 + ['*'] + list2, False]


def main():
    name1 = input("Enter the name of the first person: ").lower().replace(" ", "")
    name2 = input("Enter the name of the second person: ").lower().replace(" ", "")

    list1 = list(name1)
    list2 = list(name2)

    proceed = True
    while proceed:
        result, proceed = remove_matching_letter(list1, list2)

    count = len(list1) + len(list2)
    result = ["Friends", "Love", "Affection", "Marriage", "Enemy", "Siblings"]

    split_index = (count % len(result)) - 1

    if split_index >= 0:
        right_half = result[split_index + 1:]
        left_half = result[:split_index]
```

```python
    new_result = right_half + left_half
else:
    new_result = result[:split_index]  # Ignore the last element

while len(new_result) > 1:
    count = len(new_result)
    split_index = (count % len(new_result)) - 1

    if split_index >= 0:
        right_half = new_result[split_index + 1:]
        left_half = new_result[:split_index]
        new_result = right_half + left_half
    else:
        new_result = new_result[:split_index]  # Ignore the last element

final_result = new_result[0]
print(f"The relationship between {name1} and {name2} is: {final_result}")
```

1) What is the **primary purpose** of the function remove_matching_letter? ***1 point***

- ○ To count common letters between two names
- ○ To remove only duplicate letters from the first list
- ◉ To remove the first matching character from both lists and signal continuation
- ○ To concatenate both lists using *

Assessment submitted.
X

2) Why does the function remove_matching_letter return immediately after removing one matching character? ***1 point***

- To reduce time complexity
- To restart scanning with updated lists and avoid an IndexError
- To avoid index errors only
- To preserve list order

3) What does the proceed variable control in the while loop? ***1 point***

- Whether both names are non-empty
- Whether the result list has more than one element
- Whether a matching character was found in the previous iteration
- Whether user input is valid

4) Why are spaces removed and characters converted to lowercase before processing? ***1 point***

- To improve execution speed
- To prevent runtime errors
- To standardize input for fair character comparison
- To reduce memory usage

5) What does the value : ***1 point***

```
count = len(list1) + len(list2)
```

represent in the program?

- The total number of unmatched characters remaining after elimination
- The total number of iterations executed
- The length of the longer name
- The number of matching characters removed

6) What is the purpose of the expression ***1 point***

```
split_index = (count % len(result)) - 1
```

in the relationship elimination logic?

- To calculate the index at which the list should be rotated or reduced
- To sort the relationship list
- To randomly select a relationship
- To check whether the list length is even or odd

7) Why does the code handle the case when split_index < 0 separately? ***1 point***

- Because Python does not allow negative indices
- Because list slicing fails for negative numbers
- Because it indicates an invalid count value

Assessment submitted.
X

- [x] Because an index of -1 breaks the slicing logic used for removal

8) What ensures that the relationship elimination continues until only one option remains? ***1 point***

- [ ] The for loop inside remove_matching_letter
- [ ] The condition if split_index >= 0
- [ ] The calculation of count
- [x] The loop condition while len(new_result) > 1

9) What would happen if the first return statement (inside the if condition) were removed? ***1 point***

- [ ] The program would crash with an IndexError
- [ ] Only the first name would be processed
- [x] Multiple matches would be removed in one iteration
- [ ] The program would raise a syntax error

10) Which algorithmic pattern best describes the **relationship elimination phase** of the program? ***1 point***

- [ ] Divide and Conquer
- [ ] Greedy selection
- [ ] Backtracking
- [x] Circular counting with iterative elimination

Grayscale Image Quantization Using Python (PIL & NumPy)

Background

A small imaging startup is working on reducing the storage size of grayscale images used in low-bandwidth applications such as remote sensing dashboards and embedded systems. The goal is **not** to compress images using standard formats, but to **reduce the number of intensity levels** while preserving the overall structure of the
image.

An intern is asked to prototype a **manual grayscale quantization algorithm** using Python so that the logic is transparent and easy to reason about for learning and analysis.

Problem Statement

Given an 8-bit grayscale image (pixel values ranging from 0 to 255), reduce the image to **8 discrete gray levels** by mapping ranges of pixel intensities to a smaller set of values.

Each new pixel value should represent a **bucket** of intensities rather than the original fine-grained value.

Tools Used

- **PIL (Python Imaging Library):** for loading, converting, and saving images
- **NumPy:** for converting the processed image into an array for further analysis

Step-by-Step Explanation of the Approach

Step 1: Load and Prepare the Image

Assessment submitted.
X

- The image is loaded from disk.
- It is explicitly converted into **grayscale mode ('L')**, ensuring each pixel has a single intensity value between 0 and 255.
- A pixel access object (pixelMap) is created to read pixel values.

Step 2: Create a New Output Image

- A new grayscale image with the **same dimensions** as the original is created.
- Another pixel access object (pixelNew) is used to write the quantized pixel values.

Step 3: Quantization Logic

- Two nested loops traverse every pixel (i, j) in the image.
- Each pixel value is checked against predefined intensity ranges.
- Based on the range it falls into, the pixel is assigned a new value from **0 to 7**.

Original Intensity Range	New Value
0 – 31	0
32 – 63	1
64 – 95	2
96 – 127	3
128 – 159	4
160 – 191	5
192 – 223	6
224 – 255	7

Assessment submitted.
X

```python
from PIL import Image
import numpy

# Step 1: Load the image in grayscale mode
im = Image.open('images.jpg').convert('L')  # Ensure grayscale
pixelMap = im.load()

# Step 2: Create a new grayscale image
img = Image.new('L', im.size)
pixelNew = img.load()

# Step 3: Quantize
for i in range(img.size[0]):
    for j in range(img.size[1]):
        if 0 <= pixelMap[i, j] <= 31:
            pixelNew[i, j] = 0
        elif 32 <= pixelMap[i, j] <= 63:
            pixelNew[i, j] = 1
        elif 64 <= pixelMap[i, j] <= 95:
            pixelNew[i, j] = 2
        elif 96 <= pixelMap[i, j] <= 127:
            pixelNew[i, j] = 3
        elif 128 <= pixelMap[i, j] <= 159:
            pixelNew[i, j] = 4
        elif 160 <= pixelMap[i, j] <= 191:
            pixelNew[i, j] = 5
        elif 192 <= pixelMap[i, j] <= 223:
            pixelNew[i, j] = 6
        elif 224 <= pixelMap[i, j] <= 255:
            pixelNew[i, j] = 7


img.save('lena_2.jpg')
J = numpy.asarray(Image.open('lena_2.jpg'))
```

11) Assume a pixel at position (10, 20) in the original image has a grayscale value of **31**. ***1 point***
What will be the value stored at the same position in pixelNew?

- (●) 0
- ( ) 1
- ( ) 2
- ( ) 7

12) What will be the quantized value for a pixel whose original grayscale intensity is **223**? ***1 point***

- (●) 6
- ( ) 5
- ( ) 7
- ( ) 4

13) Assuming the input is an RGB image, what happens without grayscale conversion? ***1 point***

- ( ) The output image would still be correct
- ( ) The image would automatically convert to grayscale
- ( ) The quantization ranges would adjust dynamically
- (●) The code would fail because pixelMap would return tuples instead of single values

Assessment submitted.
X

14) Which change would **reduce execution time significantly** while keeping the output logically equivalent? ***1 point***

- Add more conditional checks
- Increase the number of intensity ranges
- Save the image in PNG instead of JPG
- Replace nested loops with NumPy vectorized operations

15) A Python program processes a grayscale image by examining each pixel value (ranging from 0 to 255). Based on fixed intensity intervals, the program assigns **one of a small set of integer values** to a new image. The transformed image is then saved and viewed using a standard image viewer with **no contrast stretching or enhancement applied**. ***1 point***

Which option best describes the visual appearance of the output image and the mathematical operation being performed on each pixel?

- The image appears nearly black with very low contrast.
  Formula:
  Pixel_new = Pixel_old / 255
- The image preserves smooth gradients but with reduced brightness.
  Formula:
  Pixel_new = Pixel_old − 32
- The image shows random speckle noise due to heavy quantization.
  Formula:
  Pixel_new = Pixel_old XOR 32
- The image appears nearly black.
  Formula:
  Pixel_new = floor(Pixel_old / 32)

You may submit any number of times before the due date. The final submission will be considered for grading.

Submit Answers